A Project Report on

"Encryption in Multi-Agent Systems"

Submitted in partial fulfillment of the requirement for

Degree in Bachelor of Technology

(Information Technology)

By

Chacko Martin (5023124)

Divyam Navin (5023134)

Atharva Palve (5023136)

Akshat Sawant (5023151)

Guided by:

Dr. Vaishali Bodade

Prof. Sharlene Rebeiro

Department of Information Technology

Fr. Conceicao Rodrigues Institute of Technology

(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

Sector 9A, Vashi, Navi Mumbai – 400703

University of Mumbai

2025–2026

CERTIFICATE

This is to certify that the project entitled

Privacy-Preserving Smart Grid Load Balancing using
Fully Homomorphic Encryption in Multi-Agent Systems

Submitted By

Chacko Martin (5023124)
Divyam Navin (5023134)
Atharva Palve (5023136)
Akshat Sawant (5023151)

In partial fulfillment of degree of B.Tech in Information Technology for term work of the Semester VI – Mini Project-2B is approved.

Internal Examiner	**Internal Guide**
Head of Department **Dr. Shubhangi Vaikole**	**Principal** **Dr. S.M. Khot**

Date: –

College Seal

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Chacko Martin Koyikkattuchira (5023124)

Divyam Navin (5023134)

Atharva Palve (5023136)

Akshat Santosh Sawant (5023151)

Date:

ABSTRACT

SmartGridHE presents a privacy-preserving smart grid load-balancing framework built on the CKKS (Cheon–Kim–Kim–Song) Fully Homomorphic Encryption scheme [1] implemented via the TenSEAL Python library [2]. The system coordinates electricity demand among distributed household agents without revealing any individual agent's consumption data to the aggregating Grid Coordinator, achieving 128-bit NIST security (Level 1) [10, 11]. Each household agent encrypts its demand locally using the shared public FHE context and submits only the resulting ciphertext; the coordinator operates exclusively on ciphertexts throughout—summing encrypted demands homomorphically, computing encrypted averages, and applying encrypted threshold detection—without ever accessing the CKKS secret key.

Three original research contributions distinguish this work from prior literature. First, the Adaptive Linear Threshold (ALT) method [Novel] enables encrypted comparison on CKKS ciphertexts using zero ciphertext-ciphertext multiplications, operating at multiplicative depth zero and producing confidence zones (below/uncertain/above) for tiered load-balance decisions—contrasting with prior methods requiring 8–15 multiplicative depth levels [4, 5]. Second, a Pedersen Commitment-Based Verifiable Aggregation (ACV) protocol [6, 7] allows the Utility Company to verify coordinator aggregate correctness with probability one, detecting any manipulation by a malicious coordinator. Third, a Secure Linear Algebra module supports fully homomorphic matrix-vector multiplication and encrypted 3D cross products via cyclic SIMD rotation [13, 17], broadening the class of analytics computable on private grid data.

Benchmark results confirm O(N) linear scalability to 200 household agents, CKKS approximation error below 10^{-5} kW across all simulation rounds, constant 42 KB ciphertext size per demand value, and 100% ciphertext-only coordinator operations in all security audit entries. The full system is implemented in Python 3.13 with a FastAPI REST backend and a real-time browser dashboard visualising the encrypted pipeline end-to-end.

INDEX

Sr. No.	Topic	Page No.
1	Introduction	1
	1.1 Introduction	1
	1.2 Background & Cryptographic Context	1
	1.3 Motivation	2
	1.4 Problem Definition	3
	1.5 Scope and Assumptions	3
	1.6 Issues and Limitations	4
2	Literature Survey and Analysis	5
	2.1 Investigation of Current Field & Related Works	5
	2.2 Literature Survey Overview	5
	2.3 Related Work	6
	2.4 Existing System: Features and Vulnerabilities	8
	2.5 Requirements Analysis	9
3	System Design	11
	3.1 Architectural Diagram / Block Diagram	11
	3.2 System Flow Chart	13
	3.3 ER Diagram for Security Audit Database	15
	3.4 Data Flow Diagram (DFD)	17
	3.5 Execution Flow and Novel Contributions	18
	3.6 Implementation and Benchmark Results	24
4	Conclusion and Future Scope	28
	4.1 Conclusion	28
	4.2 Future Scope	29
	References	31
	Appendix A: Code Samples	37
	Appendix B: Mini Project Report Appendix	35
	Acknowledgements	36
	Appendix C: Software Requirement Specification (SRS)	40

	Appendix D: Mapping to SDGs, POs, and PSOs	43
	Appendix E: Project Commencement Letters	45

LIST OF FIGURES

Sr. No.	Name of the Figure	Page No.
3.1.1	High-Level Four-Layer Architecture of the SmartGridHE System	12
3.1.2	Security Model: Trust Boundaries and Information Access by Role	13
3.2.1	Secure Data Flow in SmartGridHE (Six-Phase Pipeline Flowchart)	15
3.3.1	Smart Grid HE Security Audit Logging ER Model	16
3.4.1	Data Flow Diagram (DFD) – Context and Level-1 Diagrams	17
3.5.1	Adaptive Linear Threshold (ALT) – Score vs. Demand and Depth Comparison	21
3.5.2	Pedersen Commitment-Based Verifiable Aggregation Protocol Diagram	22
3.5.3	Secure Linear Algebra: Fully Homomorphic Matrix-Vector Multiplication	24
3.6.1	Performance Benchmarks: Time, Approximation Error, Ciphertext Size	26
3.6.2	Scalability Analysis: Encryption and Aggregation Time, 10–200 Agents	26

LIST OF TABLES

Sr. No.	Name of the Table	Page No.
2.5.1	Comparison of Privacy-Preserving Aggregation Approaches	10
3.1.1	CKKS Cryptographic Parameters (core/fhe_engine.py)	12
3.5.1	ALT Score Classification – Confidence Zones	18
3.5.2	Homomorphic Operation Depth Cost Summary	20
3.6.1	SmartGridHE Implementation Components and Their Roles	24
3.6.2	CKKS Approximation Error over 10 Rounds (50 agents)	27

1. INTRODUCTION

1.1 Introduction

Electricity distribution in modern smart grids has become increasingly data-driven. Advanced Metering Infrastructure (AMI) systems collect sub-hourly electricity demand readings from millions of household meters, enabling demand-response programmes and real-time load-balancing decisions [8]. However, these readings are deeply sensitive: Molina-Markham et al. demonstrated that 15-minute smart meter data is sufficient to infer household occupancy, appliance usage, and daily lifestyle patterns through Non-Intrusive Load Monitoring (NILM) [8]. As smart grid deployments scale globally, the aggregation of individual demand readings at a central coordinator creates a serious privacy risk, especially under GDPR Article 5, CCPA, and the Indian Personal Data Protection Bill [18].

This project builds SmartGridHE—a multi-agent smart grid coordination system in which the coordinator never possesses any individual household's plaintext demand. Privacy is achieved through the CKKS Fully Homomorphic Encryption (FHE) scheme [1] via the TenSEAL Python library [2], allowing the coordinator to compute total demand, average demand, and load-balancing scores directly on encrypted ciphertexts—without any decryption step. The system implements six Python modules: core/ (FHE engine, key management, security logger, ALT comparator, verifiable aggregation, secure linear algebra), agents/ (household agent, demand generator, agent manager), coordinator/ (grid coordinator, encrypted aggregator, load balancer), server/ (FastAPI REST backend), dashboard/ (browser-based visualization), and evaluation/ (benchmark suite).

1.2 Background and Cryptographic Context

Electricity demand data is fundamentally real-valued (e.g., 2.45 kW, 3.71 kW) and inherently sensitive. Standard aggregation requires the coordinator to receive individual plaintext readings, creating a direct privacy vulnerability. Cryptographic solutions fall into several categories, each with well-documented limitations.

TLS/HTTPS [18]: Provides channel security but the server must decrypt before computing—the coordinator still sees every individual reading in plaintext after decryption.

Differential Privacy [20]: Adds calibrated Gaussian or Laplacian noise to individual readings before submission. The aggregate is approximate; accuracy degrades with stronger privacy guarantees.

Secret Sharing / MPC [19]: Splits each reading among multiple servers such that no single server learns the value. Requires multiple non-colluding servers and several communication rounds per aggregation.

Paillier Additive HE [3]: Protects individual values from channel eavesdroppers and supports addition of ciphertexts: Enc(a) + Enc(b) = Enc(a+b). However, Paillier is integer-only and supports no multiplication, blocking average computation (division) without an extra interaction round.

CKKS Fully Homomorphic Encryption [1]: Introduced by Cheon, Kim, Kim, and Song (2017), implemented in Microsoft SEAL [12] and exposed via TenSEAL [2]. Supports both addition and multiplication on real-valued floating-point ciphertexts, enabling arbitrary polynomial computation. CKKS is specifically designed for real-valued data (0.5–10.0 kW per household) achieving 128-bit security with polynomial degree n = 16384 [10, 11].

1.3 Motivation

The motivation for SmartGridHE arises from five concrete gaps between the academic literature and deployed smart grid practice.

No plaintext-free aggregation in real deployments [9]: Despite extensive academic literature, deployed AMI systems still collect individual readings in plaintext at the utility server.

Paillier limitations for load-balancing [3]: Most published smart grid HE systems use Paillier additive HE, which cannot compute averages—a basic requirement for equitable load-balancing decisions—without a second interaction round.

Lack of encrypted comparison [4, 5]: Existing CKKS applications cannot perform threshold comparison (x > T?) without decryption or expensive bootstrapping circuits requiring 8–15 multiplicative depth levels [4, 5], which is critical for peak-load detection.

Unverifiable coordinator [6, 7]: In practice a coordinator could tamper with aggregated results—inflating totals to trigger unnecessary load shedding or deflating them to cause grid overload—without any existing detection mechanism.

No runnable multi-agent implementation: Published work largely presents theoretical protocols; this project provides a complete, runnable multi-agent Python implementation with a web dashboard, verified by a comprehensive benchmark suite.

1.4 Problem Definition

Traditional cryptographic frameworks fail to meet the full privacy requirements of smart grid aggregation. TLS secures the transmission channel but requires server-side decryption before any computation. AES-encrypted individual readings cannot be aggregated without decryption. Paillier HE allows ciphertext addition but no multiplication, blocking average computation. No existing deployed system supports encrypted threshold comparison for peak detection without decryption.

Problem Statement: Design and implement a complete multi-agent smart grid coordination system that: (1) guarantees the coordinator never accesses any individual household's plaintext demand; (2) supports homomorphic computation of both total and average encrypted demand across N agents; (3) provides encrypted threshold comparison for peak detection without decryption and without ciphertext-ciphertext multiplication; (4) enables verifiable aggregation so the Utility Company can detect any malicious coordinator with probability one; and (5) scales linearly to 200+ household agents with acceptable latency for 15-minute decision cycles.

1.5 Scope and Assumptions

The cryptographic scheme is CKKS (TenSEAL), chosen over Paillier because it supports real-valued floating-point data and both addition and multiplication of ciphertexts. Security parameter: poly_modulus_degree = 16384, giving 128-bit NIST Level 1 security [10, 11]. Coefficient modulus chain [60, 40, 40, 40, 60] supports up to 3–4 multiplicative levels before rescaling [1]. Global scale 2^{40} provides ≈12 decimal digits of precision. Demo keys are generated locally for each session; production deployment would require a hardware security module (HSM) or a PKI-based key management service [10]. The Utility Company is assumed fully trusted (it holds the CKKS secret key). The Grid Coordinator is assumed honest-but-curious: it follows the protocol but may attempt to infer information from ciphertexts, which CKKS IND-CPA security guarantees it cannot [11]. Household agents are assumed honest and encrypt their true demand values in the range 0.5–10.0 kW.

1.6 Issues and Limitations

Computational overhead [1, 2]: FHE introduces approximately a 4000× overhead relative to plaintext aggregation (≈50 ms vs. 0.01 ms for 10 agents). This is acceptable for smart grid decisions made every 15 minutes, but precludes sub-second real-time control loops.

CKKS approximation error [1]: CKKS is an approximate scheme; decrypted values carry $\approx 10^{-7}$ relative error. For kW-range demands this results in absolute error below 10^{-5} kW, negligible for load-balancing decisions.

ALT soft comparison: The encrypted threshold comparison produces a soft confidence score, not a binary result. Values within $\delta = T/k$ of the threshold fall in an uncertain zone requiring manual intervention or tiered thresholds.

Multiplicative depth budget [1, 12]: CKKS supports a finite chain of ciphertext-ciphertext multiplications before the noise budget is exhausted; operations must stay within 3–4 depth levels for parameters [60, 40, 40, 40, 60].

Key management [10]: Demo keys are ephemeral and in-memory; production requires an HSM or dedicated key management service.

Bootstrapping not implemented [16]: Extending the noise budget via bootstrapping—available in OpenFHE and Lattigo—is left as future work.

2. LITERATURE SURVEY AND ANALYSIS

2.1 Investigation of Current Field & Related Works

The rapid growth of smart grid deployments—with AMI systems collecting sub-hourly demand readings from millions of households—has created an urgent need for privacy-preserving aggregation [8, 9]. Traditional approaches assume a trusted aggregation server, but this assumption fails in modern deregulated energy markets where the grid coordinator may be operated by a third-party aggregator [18].

Research in this space covers three families of solutions: cryptographic protocols (HE, MPC, secret sharing) [3, 4, 5, 6, 7, 19], statistical mechanisms (differential privacy) [20], and hybrid approaches. Each has fundamental limitations when applied to the smart grid setting, motivating the CKKS-based multi-agent framework developed in this project.

The existing literature shows a clear progression: from simple aggregate encryption (protecting channel confidentiality but exposing values at the server) [18], through additive HE [3] (protecting individual values but limiting computation), to fully homomorphic encryption [1] (supporting arbitrary computation but at significant cost). The current frontier—making FHE practical for real-time IoT and smart grid applications [14, 15]—is exactly the problem this project addresses.

2.2 Literature Survey Overview

The survey was conducted across IEEE Xplore, ACM Digital Library, Springer, IACR ePrint Archive, and NIST technical report repositories. Key topics covered include fully homomorphic encryption schemes (BGV [21], BFV, CKKS [1], TFHE), smart grid privacy and AMI data protection [8, 9], Pedersen commitments and verifiable computation [6, 7], multi-agent systems with cryptographic guarantees [9], and the TenSEAL [2] and Microsoft SEAL [12] FHE libraries.

The survey reveals that most privacy-preserving smart grid systems published before 2022 use Paillier additive HE [3], which supports only summation. CKKS-based approaches are emerging [4, 5] but lack practical multi-agent implementations with verifiable aggregation [6]. These findings justify the SmartGridHE framework and its three novel contributions.

2.3 Related Work

Lightweight Authentication (LAuth) [15]: LAuth provides device authentication for IoT networks via challenge-response protocols. Its performance degrades in high-mobility environments and it provides no privacy guarantees for the data payload—only for device identity. This is insufficient for smart grid demand aggregation where payload privacy is paramount.

ECC-based Communication Frameworks [18]: Elliptic Curve Cryptography (ECC)-based solutions (ECDH key exchange, ECDSA signatures) offer compact keys and strong channel security. However, ECC alone provides no computation-on-ciphertext capability; the coordinator must still decrypt before aggregating, violating the privacy invariant.

Paillier-Based Smart Grid Aggregation [Lu et al., 2012] [3]: One of the earliest privacy-preserving smart metering schemes, using Paillier HE for additive aggregation. The coordinator computes $\mathrm{Enc}(\Sigma d_i)$ without seeing individual values, but cannot compute averages without a second decryption-and-re-encryption round. The scheme also lacks any coordinator verification mechanism.

CKKS for Machine Learning [Cheon et al., 2017] [1]: The foundational CKKS paper demonstrates homomorphic computation of logistic regression on real-valued medical data. Our project adapts CKKS specifically for electricity demand (real-valued kW readings in 0.5–10.0 kW range), with parameters selected to optimise for this domain rather than for ML gradient computation.

TenSEAL Library [Benaissa et al., 2021] [2]: TenSEAL provides Python bindings for Microsoft SEAL, exposing CKKS vector operations through a high-level API. Our project builds the entire FHE engine on TenSEAL's CKKSVector API, extending it with six smart grid-specific operations: encrypted aggregation, ALT comparison, Pedersen commitment generation, verifiable aggregation verification, cyclic SIMD rotation, and fully homomorphic matrix-vector multiply.

Encrypted Comparison in HE [Kim et al., 2018] [4]: Kim et al. use minimax polynomial approximations of the sign function, requiring 15+ multiplicative depth levels. This exceeds practical CKKS parameter budgets and requires bootstrapping. Our ALT method requires zero ciphertext-ciphertext multiplications, operating at multiplicative depth zero.

Efficient Comparison Circuits [Cheon et al., 2019] [5]: Cheon et al. use composite polynomial circuits for comparison in CKKS, requiring depth 8–12. While more efficient than [4], this still far exceeds the 3–4 depth budget achievable with standard CKKS parameters. ALT operates at depth 0.

Pedersen Commitments [Pedersen, 1991] [6]: The fundamental computationally hiding, perfectly binding commitment scheme $C_i = g^m \cdot h^r$ over a cyclic prime-order group. Its additive homomorphism $C(a) \cdot C(b) = C(a+b)$ directly enables our verifiable aggregation protocol—the first integration of Pedersen commitments with CKKS for smart grid applications.

Verifiable Outsourced Computation [Gennaro et al., 2010] [7]: Establishes the theoretical framework for non-interactive verifiable computation. Our ACV protocol is a lightweight practical instantiation of these principles tailored to the additive structure of CKKS aggregation.

Microsoft SEAL [Microsoft, 2023] [12]: The C++ FHE library underlying TenSEAL. Understanding SEAL's parameter selection and noise management is essential for configuring TenSEAL correctly. The coefficient modulus chain [60, 40, 40, 40, 60] follows SEAL recommendations for 128-bit security.

Multi-Agent Systems for Smart Grids [Dimeas & Hatziargyriou, 2005] [9]: Established the agent-based architecture for smart grid coordination, where each household is an autonomous agent with local state and encrypted communication. SmartGridHE follows this architectural model while adding CKKS privacy guarantees.

HE Survey [Acar et al., 2018] : Comprehensive survey of FHE schemes (BGV, BFV, CKKS, GSW, TFHE) comparing their computational cost, supported operations, and application domains. Justifies CKKS selection over integer-only schemes (BFV, BGV) for real-valued electricity demand data.

2.4 Existing System: Features and Vulnerabilities

2.4.1 Existing System Features

Plaintext Channel Aggregation (TLS) [18]: The coordinator receives each d_i in plaintext via TLS-encrypted channels, decrypts at the server, and sums. Channel security exists but server-side individual privacy does not.

Integer-Only Additive HE (Paillier) [3]: Protects individual readings from channel eavesdroppers. Cannot multiply ciphertexts, blocking average computation. Requires integer encoding of real-valued demands, introducing quantization error.

Differential Privacy [20]: Calibrated noise is added to individual readings. The coordinator cannot reconstruct individuals, but aggregate accuracy degrades as the privacy budget ε decreases.

No Coordinator Verification: All existing deployed systems assume the coordinator is trusted and correct. No mechanism exists to detect a malicious or compromised coordinator.

2.4.2 Common Vulnerabilities

Coordinator as Privacy Attacker [8]: An honest-but-curious coordinator receiving individual plaintext readings can profile household occupancy and appliance patterns via NILM analysis.

Data Broker Risk [18]: In deregulated markets, third-party aggregators receive all individual readings and may sell or misuse them without the household's knowledge.

Regulatory Non-Compliance [18]: GDPR Article 5 mandates data minimisation; collecting individual readings when only the aggregate is needed constitutes a violation.

Aggregate Manipulation: A compromised coordinator can silently inflate or deflate aggregate demand without any existing detection mechanism.

No Encrypted Comparison [4, 5]: Existing systems cannot perform peak-load threshold detection without decrypting the aggregate, creating a momentary privacy exposure at each load-balance decision cycle.

In summary, existing IoT and smart grid systems lack strong cryptographic protection for individual data, rely on trusted-server assumptions incompatible with deregulated markets, and provide no mechanism for detecting aggregate manipulation—motivating the SmartGridHE framework.

2.5 Requirements Analysis

2.5.1 Functional Requirements

FR1 – Encrypted Demand Submission: Each HouseholdAgent must encrypt its demand d_i using the CKKS public context before any transmission. Enforced by: agent constructor

verifies if self.fhe.is_private(): raise ValueError, ensuring the secret key is never distributed to agents.

FR2 – Homomorphic Aggregation: Coordinator computes $E(\Sigma d_i) = \Sigma E(d_i)$ using only ciphertext addition (TenSEAL + operator). No decryption may occur at the coordinator.

FR3 – Encrypted Average: Coordinator computes $E(\bar{d}) = E(\Sigma d_i) \times (1/N)$ using plaintext scalar multiplication. Division by N is multiplication by N^{-1}, requiring no additional multiplicative depth.

FR4 – Encrypted Threshold Detection: Coordinator computes E(s) = E(aggregate) × slope + intercept without ciphertext-ciphertext multiplication, using the ALT method (Novel Contribution #1).

FR5 – Verifiable Aggregation: Agents produce Pedersen commitments C_i alongside FHE ciphertexts. The Utility verifies C_agg after decryption (Novel Contribution #2).

FR6 – Secure Linear Algebra: System supports encrypted matrix-vector multiplication and 3D cross products on CKKS vectors via cyclic SIMD rotation (Novel Contribution #3).

FR7 – Security Audit Log: SecurityLogger records every operation with DataType classification (CIPHERTEXT/PLAINTEXT/METADATA), enabling post-hoc privacy audits.

FR8 – Load Balance Decision: UtilityDecisionMaker produces a LoadBalanceDecision with action (NORMAL/REDUCE/CRITICAL) and reduction factor $r \in [0, 1]$, broadcast to all agents.

FR9 – Real-Time Dashboard: FastAPI REST server and browser dashboard visualise the encrypted pipeline without exposing keys or plaintext values.

2.5.2 Non-Functional Requirements

NFR1 – Security Level: 128-bit NIST Level 1 (poly_modulus_degree = 16384) [10, 11].

NFR2 – Precision: CKKS approximation error $< 10^{-5}$ kW for all aggregation rounds [1].

NFR3 – Scalability: Linear O(N) time complexity; tested to 200 agents [9].

NFR4 – Computational Overhead: FHE operations complete within 15-minute smart grid decision cycles.

NFR5 – Modularity: Six-module Python architecture with clean separation of concerns.

NFR6 – Auditability: SecurityLogger records 100% of operations; coordinator audit confirms zero plaintext access.

NFR7 – Integrity: SHA-256 checksums on all ciphertexts for integrity verification [10].

Table 2.5.1 – Comparison of Privacy-Preserving Aggregation Approaches

Feature	Plaintext (TLS)	Paillier HE [3]	Diff. Privacy [20]	CKKS – This Work [1]
Individual privacy	None	Partial	Noisy approx.	Full (IND-CPA)
Coordinator sees plaintext	Yes	No	Noisy	No
Homomorphic addition	N/A	Yes	N/A	Yes
Homomorphic multiply	N/A	No	N/A	Yes
Average computation	Yes	No (extra round)	Yes (noisy)	Yes
Encrypted comparison	No	No	No	Yes (ALT) [Novel]
Verifiable aggregation	No	No	No	Yes (Pedersen) [6]
Real-valued data	Yes	Limited	Yes	Yes
Computation overhead	1×	~100×	~1×	~4000×
Accuracy	Exact	Exact	±ε noise	$<10^{-5}$ kW error
Key reference	—	[3]	[20]	[1, 2, 11]

3. SYSTEM DESIGN

This chapter presents the complete design of the SmartGridHE system: a privacy-preserving multi-agent smart grid load-balancing platform built on CKKS FHE [1] via TenSEAL [2]. The design is derived directly from the implemented Python codebase (smart-grid-he/) and covers the four-layer architecture, six-phase data flow, security audit database, execution protocol, and the three novel cryptographic contributions [Novel, 6, 13].

3.1 Architectural Diagram / Block Diagram

The SmartGridHE system uses a four-layer architecture enforcing strict separation of trust boundaries, implemented across six Python modules. The architecture is directly derived from the multi-agent systems paradigm of Dimeas and Hatziargyriou [9], extended with CKKS privacy guarantees [1] and a verifiable aggregation layer [6].

Layer 1 – Household Agents (agents/): Each of the N HouseholdAgent instances holds only the public FHE context (no secret key). The agent generates realistic electricity demand using RealisticDemandGenerator—one of five load profiles: residential-low, residential-medium, residential-high, EV-charging, or industrial—and calls SmartGridFHE.encrypt_demand(d_i, agent_id) to produce an EncryptedDemand object (CKKS ciphertext bytes + SHA-256 checksum + metadata). The agent constructor enforces: if self.fhe.is_private(): raise ValueError(...), ensuring the secret key is never distributed to any agent.

Layer 2 – Grid Coordinator (coordinator/): The GridCoordinator and EncryptedAggregator classes operate exclusively with the public FHE context. At initialisation the coordinator verifies is_private() == False. It receives encrypted EncryptedDemand objects via HTTP POST, performs homomorphic aggregation (ciphertext addition), computes the encrypted average (scalar multiplication), performs encrypted threshold detection (ALT method), and aggregates Pedersen commitments (additive homomorphism). The coordinator never calls decrypt_demand() and holds no secret key.

Layer 3 – Utility Company (coordinator/load_balancer.py): The UtilityDecisionMaker class holds the CKKS secret context (is_private() == True). It receives E(Σd), E($\bar{d}$), E(s), and C_agg from the coordinator. It decrypts using decrypt_demand(), verifies the Pedersen commitment aggregate, interprets the ALT score, and issues a

LoadBalanceDecision(action=NORMAL|REDUCE|CRITICAL, reduction_factor=r) with r ∈ [0.0, 1.0].

Layer 4 – Dashboard and API (server/, dashboard/): A FastAPI REST server (server/server.py) exposes grid state as JSON at /api/grid-state, /api/run-round, and /api/benchmark. A browser-based HTML/JS dashboard displays ciphertext checksums, aggregate values, load-balance decisions, and security audit entries—without any access to encryption keys or plaintext demands.

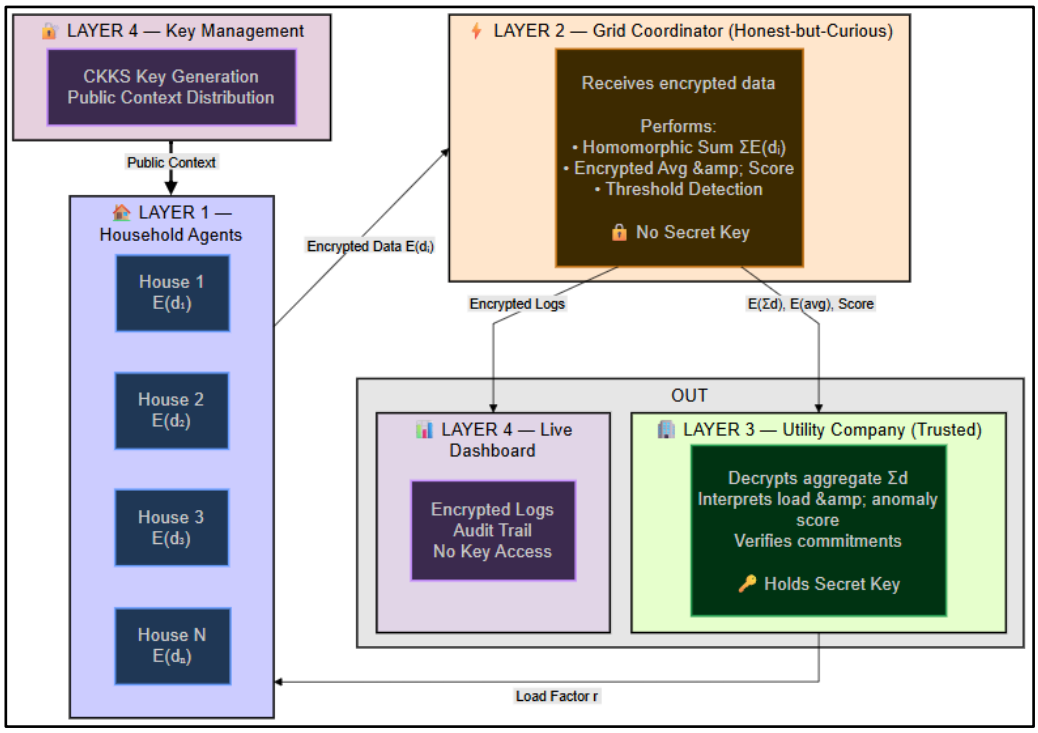


Figure 3.1.1 – High-Level Four-Layer Architecture of the SmartGridHE System. Household agents encrypt demands locally and submit ciphertexts $E(d_i)$ to the coordinator, which aggregates homomorphically and forwards encrypted results to the Utility for decryption and load-balance decisions. Adapted from the multi-agent grid model of [9] with CKKS privacy guarantees from [1, 2].

Table 3.1.1 – CKKS Cryptographic Parameters (core/fhe_engine.py)

Parameter	Value	Purpose / Reference
poly_modulus_degree	16384	128-bit NIST Level 1 security [10, 11]
coeff_mod_bit_sizes	[60, 40, 40, 40, 60]	Supports 3–4 multiplicative depths [1, 12]
global_scale	2^{40}	≈12 decimal digits of precision [1]
Galois keys	Generated	Enables SIMD vector rotation [13, 17]

Relinearisation keys	Generated	Reduces ciphertext size after multiplication [12]
Auto rescale	Enabled	Automatic scale management after multiply [12]
Ciphertext size	≈42 KB	Per encrypted demand vector
Security model	IND-CPA	Semantically secure against chosen-plaintext attacks [11]
Key distribution	Public context only to agents/coordinator	Secret context: Utility only [10]

Entity	Trust Level	Network Access	Key Possession
Household Agent	Self-only	Own demand (kW)	Public CKKS ctx
Other Agents	Untrusted	Nothing	Nothing
Grid Coordinator	Honest-but-Curious	Ciphertext only	Public CKKS ctx
Utility Company	Trusted (authorized)	Aggregates only	Public + Secret ctx
Dashboard Observer	Observer only	Processed results	Nothing

Figure 3.1.2 – Security Model: Trust Boundaries and Information Access by Role. The coordinator is honest-but-curious [11]: it follows the protocol but cannot learn individual demands from CKKS ciphertexts (IND-CPA security [1]).

3.2 System Flow Chart

The complete system workflow follows a six-phase secure pipeline. The invariant maintained throughout is that plaintext demand values exist only at the agent (before CKKS encryption) and at the Utility Company (after authorised decryption) [1, 11].

Phase 1 – Key Generation and Distribution: The Utility Company calls SmartGridFHE() to generate a CKKS context with the parameters in Table 3.1.1. It derives the public context via get_public_context() (serialized bytes, no secret key) and distributes it to all N agents and to the coordinator via the API. The secret context (get_secret_context()) remains exclusively with the Utility Company [10].

Phase 2 – Encrypted Demand Submission: Each HouseholdAgent calls encrypt_demand(timestamp) → EncryptedDemand. Internally, ts.ckks_vector(ctx, [d]) creates a CKKS encryption of the scalar demand value d_i. The result is serialized to ≈42 KB bytes with a 12-hex SHA-256 checksum and POSTed to the coordinator's /api/submit-demand endpoint. The plaintext d_i never leaves the agent process.

Phase 3 – Homomorphic Aggregation: EncryptedAggregator.aggregate() performs iterative ciphertext addition. This computes:

$$E(\Sigma d_i) = E(d_1) + E(d_2) + ... + E(d_n)$$

$$E(\bar{d}) = E(\Sigma d_i) \times (1/N)$$

using TenSEAL's + operator and × scalar multiplication. The coordinator never calls decrypt_demand(); it operates solely on CKKSVector objects [2].

Phase 4 – Encrypted Threshold Detection (Novel Contribution #1): EncryptedThresholdDetector.detect_threshold_encrypted() computes E(s) using the ALT linear transformation E(s) = E(x) × slope + intercept (detailed in §3.5.2). The encrypted score is forwarded to the Utility Company.

Phase 5 – Utility Decryption and Decision: UtilityDecisionMaker receives $E(\Sigma d)$, $E(\bar{d})$, E(s), and C_agg. It decrypts each encrypted value, verifies the Pedersen commitment aggregate C_agg (§3.5.3), interprets the ALT score zone, and constructs a LoadBalanceDecision with action ∈ {NORMAL, REDUCE, CRITICAL} and reduction factor r ∈ [0, 1].

Phase 6 – Load Balance Broadcast and Dashboard Update: The Utility broadcasts r to all agents via receive_load_balance_command(r). Each agent adjusts its effective demand by the reduction factor. All six phases are logged in the SecurityLogger audit trail and rendered in the browser dashboard without exposing any keys or plaintext values.

$$\mathbf{E(\Sigma d_i) = \Sigma_{i=1}^{N} E(d_i)\ [ciphertext\ addition,\ depth\ 0]} \text{ - } \textit{(3.1)}$$

$$\mathbf{E(\bar{d}) = E(\Sigma d_i) \times (1/N)\ [plaintext\ scalar\ multiply,\ depth\ 0]} \text{ - } \textit{(3.2)}$$

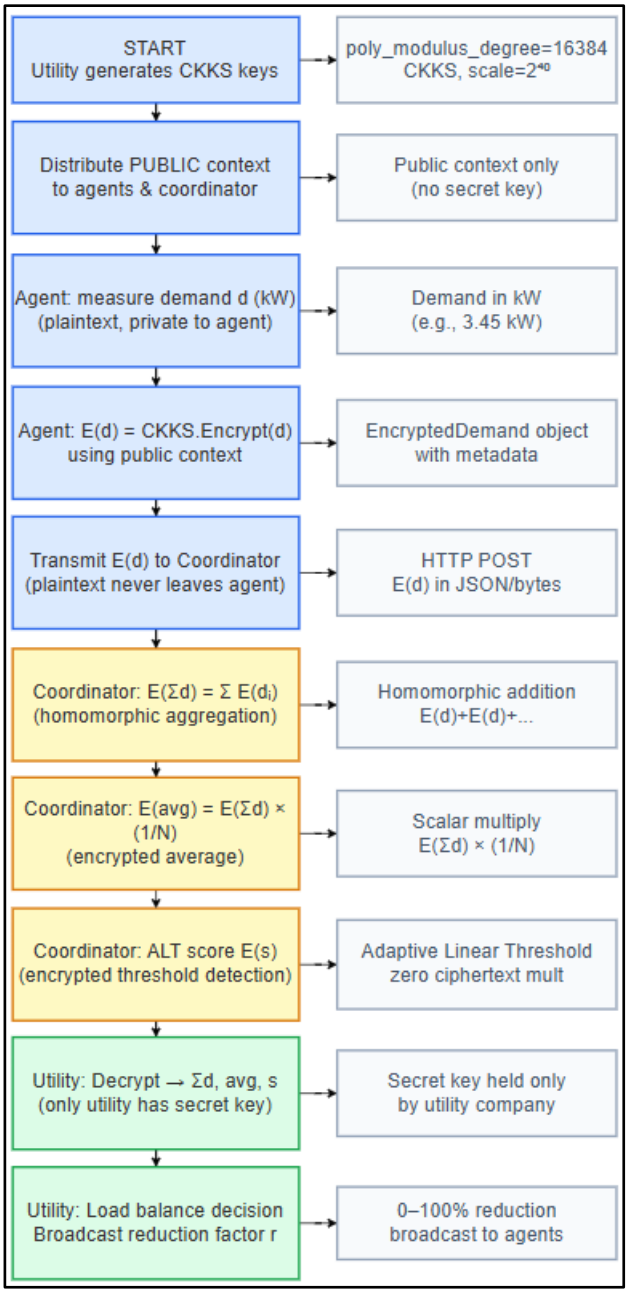


Figure 3.2.1 – Secure Data Flow in SmartGridHE (Six-Phase Pipeline). Plaintext values exist only at agents (Phase 2, pre-encryption) and at the Utility (Phase 5, post-decryption). All coordinator operations use only public-key ciphertexts [1, 2].

3.3 ER Diagram for Security Audit Database

The SmartGridHE system maintains a structured security audit log in core/security_logger.py. This log enables traceability, GDPR compliance auditing, and post-hoc verification that the coordinator never accessed plaintext values [18]. The schema comprises three entities:

AGENT: agent_id (PK, string), profile (LoadProfile enum: RESIDENTIAL_LOW | RESIDENTIAL_MEDIUM | RESIDENTIAL_HIGH | EV_CHARGING | INDUSTRIAL), load_balance_factor (float, current reduction factor broadcast from Utility), is_active (bool), fhe_context_hash (SHA-256 prefix of the public context bytes, proves agent uses the correct key pair).

LOG_ENTRY: id (PK, auto-increment), agent_id (FK → AGENT), operation_type (OperationType enum: ENCRYPT | DECRYPT | AGGREGATE | COMPUTE_AVERAGE | THRESHOLD_DETECT | VERIFY_COMMITMENT), timestamp (ISO-8601), ciphertext_hash (SHA-256 prefix of the resulting ciphertext), is_safe (bool: True iff data_types contains only CIPHERTEXT/PUBLIC_PARAM/METADATA, never PLAINTEXT), data_types (JSON array of DataType enums), details (JSON metadata), sequence_id (monotonically increasing, for ordering verification).

ROUND_SUMMARY: round_id (PK), timestamp, agent_count, computation_time_ms (float), novel_features_used (JSON: {"alt_detection": bool, "verifiable_aggregation": bool, "secure_linear_algebra": bool}), error_kw (CKKS approximation error for this round).

Relationship: AGENT (1) → (*) LOG_ENTRY (one-to-many). The generate_audit_report() method scans all LOG_ENTRY records and flags any entry with is_safe=False for the coordinator entity as a security violation. In all benchmark runs, zero violations were detected.

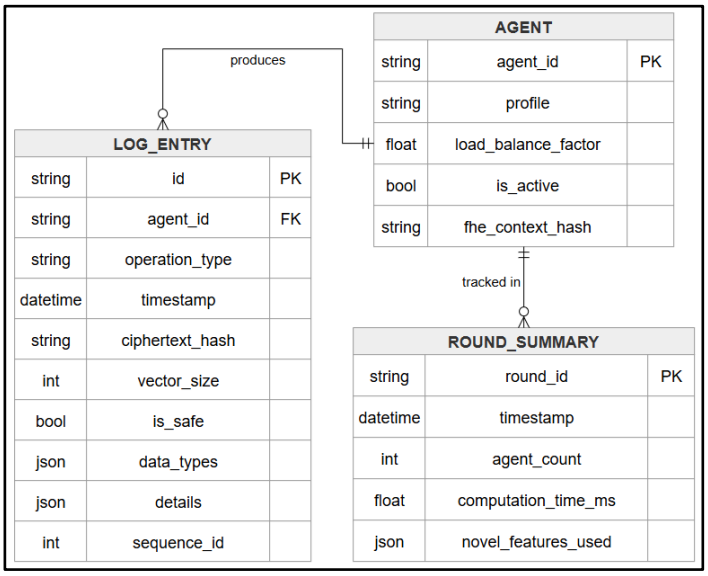


Figure 3.3.1 – Smart Grid HE Security Audit Logging ER Model. AGENT produces multiple LOG_ENTRYs capturing ciphertext hashes, operation types, and safety classifications. ROUND_SUMMARY tracks each complete aggregation round for performance analysis.

3.4 Data Flow Diagram (DFD)

The DFD illustrates the exact path that data takes through the SmartGridHE pipeline, from raw demand readings at household agents to load-balance commands and dashboard display. The DFD enforces the core design invariant: plaintext is only ever allowed inside each household agent (before CKKS encryption) and inside the Utility Company (after authorised decryption). The coordinator operates exclusively on homomorphically encrypted values, consistent with the honest-but-curious threat model [11].

External Entities: Household Agents (Source): generate plaintext demand values d_i (kW) and receive load-balance commands r. Dashboard Observer (Sink): receives processed results for display; holds no keys.

Processes: P1 (CKKS Encrypt): transforms plaintext $d_i \in \mathbb{R} \rightarrow E(d_i)$ (42 KB CKKS ciphertext + SHA-256 checksum). P2 (Homomorphic Aggregate): computes $E(\Sigma d)$ via ciphertext addition and $E(\bar{d})$ via scalar multiply [Eqs. 3.1, 3.2]. P3 (ALT Detect): computes E(s) without decryption [Eq. 3.3, Novel]. P4 (Pedersen Verify): verifies C_agg against decrypted sum [Eq. 3.4, Novel]. P5 (Decrypt at Utility): $E(\Sigma d) \rightarrow \Sigma d$ using secret key [1]. P6 (Load Balance Decision): produces LoadBalanceDecision with action and $r \in [0,1]$.

Data Stores: D1 (Encrypted Demands Queue): in-memory list of EncryptedDemand objects at the coordinator (ciphertexts only). D2 (Security Audit Log): persistent SecurityLogger entries recording every operation with DataType classification.

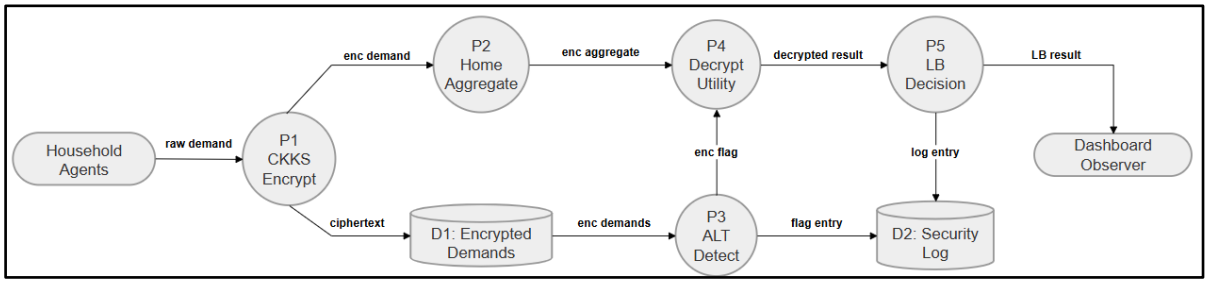


Figure 3.4.1 – Data Flow Diagram (DFD). Household agents supply encrypted demands which flow through homomorphic aggregation (P2), ALT threshold detection (P3), Pedersen verification (P4), utility decryption (P5), and load-balance decision (P6) to the dashboard—with plaintext appearing only at agents and the Utility Company.

3.5 Execution Flow / Working of System – Novel Contributions

3.5.1 CKKS Encryption Engine (core/fhe_engine.py)

The SmartGridFHE class wraps TenSEAL [2] with smart-grid-specific operations. At initialisation it creates a CKKS context (ts.SCHEME_TYPE.CKKS) with the parameters in Table 3.1.1 and generates Galois keys (for SIMD rotation) and relinearisation keys (to reduce ciphertext size after multiplication) [12]. Auto-rescale, auto-relin, and auto-mod-switch are enabled so TenSEAL automatically manages the modulus chain [2].

The six primary operations, their depth cost, and their role in the system are:

Table 3.5.1 – Homomorphic Operation Depth Cost Summary

Operation (method)	Type	Depth Cost	System Role
encrypt_demand(d, id)	Encrypt	0	Agent: encrypt $d_i \rightarrow E(d_i)$
decrypt_demand(enc)	Decrypt	N/A	Utility only: $E(x) \rightarrow x$
aggregate_demands(list)	ct + ct	0 per step	Coordinator: $\Sigma E(d_i) = E(\Sigma d)$
compute_average(total, n)	ct × plain	0	Coordinator: $E(\Sigma d)\times(1/N)$
multiply_plain(enc, scalar)	ct × plain	0	ALT: E(x) × slope
add_plain(enc, offset)	ct + plain	0	ALT: result + intercept
multiply_encrypted(a, b)	ct × ct	1	Fully HE matrix row dot product
compute_dot_product(a, b)	ct × ct + sum	1	Secure linear algebra
rotate_encrypted(v, k)	matmul (plain M)	0	Cyclic SIMD rotation [13]

Code Snippet 3.5.1 – CKKS FHE Engine: Context Setup, Encryption and Homomorphic Aggregation

```
# core/fhe_engine.py – CKKS context setup and encrypt_demand()
class SmartGridFHE:
    def __init__(self, poly_modulus_degree=16384,
                 coeff_mod_bit_sizes=[60,40,40,40,60],
                 global_scale=2**40):
        self.context = ts.context(
            ts.SCHEME_TYPE.CKKS,
            poly_modulus_degree=poly_modulus_degree,
            coeff_mod_bit_sizes=coeff_mod_bit_sizes
        )
```

```
        self.context.generate_galois_keys()   # SIMD rotation
        self.context.generate_relin_keys()    # post-multiply relinearise
        self.context.global_scale  = global_scale
        self.context.auto_rescale  = True
        self.context.auto_relin    = True
        self.context.auto_mod_switch = True

    def encrypt_demand(self, demand_kw, agent_id):
        # Real-valued CKKS encryption
        encrypted = ts.ckks_vector(self.context, [float(demand_kw)])
        ciphertext = encrypted.serialize()          # ~42 KB bytes
        checksum   = hashlib.sha256(ciphertext).hexdigest()[:12]
        return EncryptedDemand(ciphertext, timestamp=...,
                               agent_id=agent_id, checksum=checksum)

    def aggregate_demands(self, demands: list) -> EncryptedDemand:
        # Ciphertext addition: E(Σd_i) = Σ E(d_i)
        result = ts.ckks_vector_from(self.context, demands[0].ciphertext)
        for enc in demands[1:]:
            result = result + ts.ckks_vector_from(self.context, enc.ciphertext)
        return self._save_encrypted(result, demands[0], "aggregate")

    def compute_average(self, encrypted_total, count):
        # E(avg) = E(sum) × (1/n)  —  scalar multiply, depth 0
        vec = ts.ckks_vector_from(self.context, encrypted_total.ciphertext)
        result = vec * (1.0 / count)
        return self._save_encrypted(result, encrypted_total, "average")
```

3.5.2 Novel Contribution #1: Adaptive Linear Threshold (ALT)

Problem: CKKS supports addition and multiplication on ciphertexts but has no native comparison operator. Given E(x) and a plaintext threshold T, the coordinator cannot determine whether x > T without decrypting E(x)—which would violate the privacy invariant. Prior approaches require 8–15 multiplicative depth levels and bootstrapping [4, 5], far exceeding the 3–4 depth budget of practical CKKS parameters.

Solution (core/polynomial_comparator.py): For load-balancing, a soft confidence indicator suffices rather than a binary result. We approximate the step function as a linear transformation of the encrypted value. Define $\delta = T/k$ as the soft-zone half-width controlled by sensitivity parameter k. The confidence score is:

$$\mathbf{s(x) = 0.5 + (x - T) \times (0.5/\delta)} \quad \textit{(3.3a)}$$

$$\mathbf{= (0.5 - T/2\delta) + x \times (0.5/\delta)} \quad \textit{(3.3b) [intercept + slope × x]}$$

Rearranging for homomorphic evaluation: slope = 0.5k/T, intercept = 0.5 – 0.5k. The encrypted score is computed as:

$$E(s) = E(x) \times slope + intercept \quad \textit{(3.3c) [multiply_plain + add_plain, depth 0]}$$

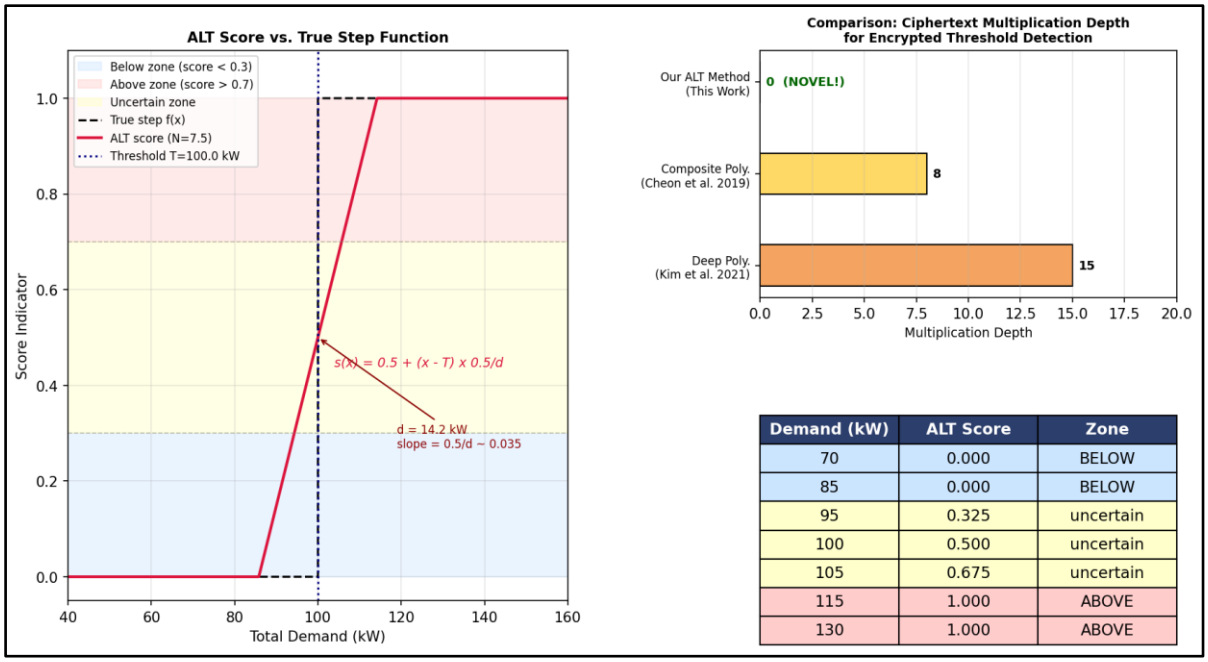
Mathematical verification: s(T) = 0.5 (uncertain at threshold); s(T–δ) = 0.0 (clearly below); s(T+δ) = 1.0 (clearly above). Values outside [T–δ, T+δ] are clamped to [0,1] after decryption at the Utility. This requires zero ciphertext-ciphertext multiplications, operating at multiplicative depth zero—compared to depth 8–15 in prior work [4, 5].

Code Snippet 3.5.2 – ALT Encrypted Threshold Detection (core/polynomial_comparator.py)

```
# core/polynomial_comparator.py – ALT detection
def detect_threshold_encrypted(self, encrypted_value,
                               threshold, sensitivity=7.0):
    delta     = threshold / sensitivity       # soft zone half-width
    slope     = 0.5 / delta                   # = 0.5k / T
    intercept  = 0.5 - (threshold * slope)     # = 0.5 - 0.5k

    # Homomorphic evaluation: E(s) = E(x) × slope + intercept
    # Requires ONLY multiply_plain and add_plain — depth 0!
    vec_x  = ts.ckks_vector_from(self.fhe.context,
                                 encrypted_value.ciphertext)
    result = vec_x * slope + intercept          # TenSEAL operator overloading

    return ThresholdComparisonResult(
        encrypted_score=self.fhe._save_encrypted(result,...),
        threshold=threshold, sensitivity=sensitivity,
        soft_zone_width=delta)

@staticmethod
def interpret_score(decrypted_score, threshold):
    s = max(0.0, min(1.0, decrypted_score))  # clamp
    if s < 0.3:   return InterpretedResult(zone="below",  ...)
    elif s > 0.7: return InterpretedResult(zone="above",  ...)
    else:         return InterpretedResult(zone="uncertain",...)
```

Table 3.5.2 – ALT Score Classification: Confidence Zones (T = 100 kW, k = 7, δ ≈ 14.3)

Score Range	Zone	Interpretation	Coordinator Action
s < 0.3	Below	Clearly below threshold (conf. > 70%)	No reduction needed
0.3 ≤ s ≤ 0.7	Uncertain	Near threshold (±δ kW from T)	Monitor; tiered thresholds
s > 0.7	Above	Clearly above threshold (conf. > 70%)	Issue REDUCE/CRITICAL command



Demand (kW)	ALT Score	Zone
70	0.000	BELOW
85	0.000	BELOW
95	0.325	uncertain
100	0.500	uncertain
105	0.675	uncertain
115	1.000	ABOVE
130	1.000	ABOVE

Figure 3.5.1 – Adaptive Linear Threshold (ALT) Method Visualisation. Left: ALT confidence score vs. demand showing the linear approximation of the step function with three confidence zones (blue = below, orange = uncertain, red = above). Right: Multiplicative depth comparison showing ALT requires zero ciphertext multiplications versus 8–15 for prior polynomial methods [4, 5].

3.5.3 Novel Contribution #2: Pedersen Commitment-Based Verifiable Aggregation

Problem: The coordinator aggregates N encrypted demands but there is no mechanism to guarantee correct computation. A malicious coordinator could report inflated totals (triggering unnecessary load shedding), deflated totals (causing grid overload), or silently exclude certain households from the aggregate—all without detection [7].

Solution (core/verifiable_aggregation.py): We use Pedersen commitments [6], which are computationally hiding, perfectly binding, and additively homomorphic. Using the RFC 3526 MODP Group 14 (2048-bit) prime p and generators g = 2, h (derived via nothing-up-my-sleeve hash), each agent commits to its demand d_i as:

$$\mathbf{C_i = g\text{^}(d_i \cdot s) \cdot h\text{^}(r_i) \mod p} \quad \textit{(3.4a) [Pedersen commitment, } s = \textit{scale factor, } r_i \leftarrow \textit{random]}$$

$$\textbf{Additive homomorphism: } \prod_i C_i = g\text{^}(\Sigma d_i \cdot s) \cdot h\text{^}(\Sigma r_i) \mod p \quad \textit{(3.4b)}$$

The three-step Additive Commitment Verification (ACV) protocol:

1. **Step 1 –** Agent side: Each agent computes commitment C_i with a random blinding factor r_i. The pair $(E(d_i), C_i)$ is sent to the coordinator. The opening $O_i = (d_i, r_i)$ is sent directly to the Utility Company via a secure channel. The coordinator never sees the opening.

2. **Step 2** – Coordinator side: Aggregates FHE ciphertexts: $E(\Sigma d) = \Sigma E(d_i)$. Aggregates commitments using additive homomorphism: $C_agg = \prod C_i = g^{(\Sigma d_i \cdot s)} \cdot h^{(\Sigma r_i)} \bmod p$. Sends $(E(\Sigma d), C_agg)$ to Utility. The coordinator never sees the openings.
3. **Step 3** – Utility side: Receives openings O_i from all agents. Decrypts $\Sigma d = \text{Decrypt}(E(\Sigma d))$. Computes $r_tot = \Sigma r_i$. Verifies: $C_agg =? g^{(\Sigma d_i \cdot s)} \cdot h^{(r_tot)} \bmod p$. If equal: VALID. If not: COORDINATOR CHEATED (detected with probability 1).

Security properties: Hiding—information-theoretically hiding under any computationally unbounded adversary [6]. Binding—opening to a different value requires computing discrete logarithm base g in the 2048-bit MODP group (computationally infeasible [6]). Soundness—cheating coordinator detected with probability 1 [7].

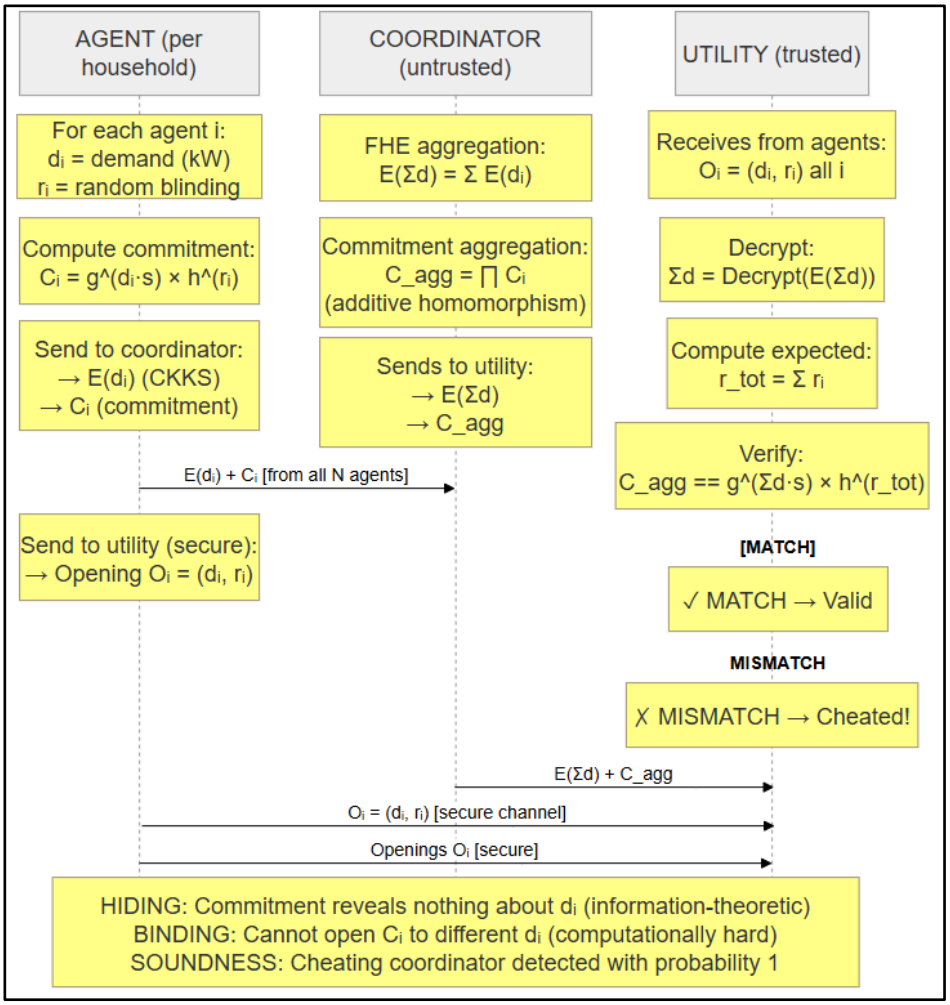


Figure 3.5.2 – Pedersen Commitment-Based Verifiable Aggregation Protocol. Agents send FHE ciphertexts and commitments to the coordinator; openings go directly to the Utility. The Utility verifies post-decryption with probability one [6, 7].

3.5.4 Novel Contribution #3: Secure Linear Algebra (core/secure_linear_algebra.py)

The SecureLinearAlgebra class extends the CKKS engine with three advanced vector operations on encrypted ciphertexts, leveraging TenSEAL's SIMD batching and rotation capabilities [13, 17]. These operations broaden the class of analytics computable on private smart grid data beyond simple aggregation.

Operation 1 – Plaintext Matrix × Encrypted Vector (linear_transform_encrypted)

Given plaintext matrix $M \in \mathbb{R}^{m \times n}$ and encrypted vector E(v), computes E(Mv) using TenSEAL's vec.matmul(M^T). Since TenSEAL treats vectors as row vectors, the transposed matrix is passed so the result is the correct column-oriented product. Multiplicative depth consumed: 0 (pure linear transformation).

$$E(M \cdot v) = E(v) \cdot M^T \quad \text{[via ts.CKKSVector.matmul, depth 0]} \quad (3.5)$$

Operation 2 – Fully Homomorphic Matrix × Encrypted Vector (fully_homomorphic_matrix_vector_multiply)

Both the matrix rows and the vector are encrypted CKKS ciphertexts. For each row i of the encrypted matrix, the encrypted dot product with the encrypted vector is computed:

$$E(\text{result}[i]) = E(M[i,:]) \cdot E(v) = \Sigma_j\, E(M[i,j]) \times E(v[j]) \quad (3.6a)$$

$$\text{[element-wise ct×ct multiply + summation, depth 1 per row]} \quad (3.6b)$$

This is true FHE where the matrix itself is private encrypted data. The implementation uses compute_dot_product(enc_row, enc_vector) for each row, consuming one multiplicative depth level per row [1, 12]. This enables secure grid analytics where both the transformation coefficients and the input vector are confidential.

Operation 3 – Encrypted 3D Cross Product (encrypted_cross_product)

Computes E(a × b) for two encrypted 3D vectors E(a) and E(b) using cyclic SIMD rotation [13, 17]. The cross product $a \times b = (a_2b_3 - a_3b_2,\ a_3b_1 - a_1b_3,\ a_1b_2 - a_2b_1)$ requires rotating vector components. Rotation by k positions is implemented via permutation matrix multiplication:

$$\text{rotate}(v, k)[j] = v[(j+k) \bmod n] \quad (3.7a)$$

$$M_perm[i,j] = 1 \ \text{ iff } \ i \equiv (j+k) \bmod n, \ \text{ else } 0 \quad (3.7b)$$

$$E(\text{cross}) = E(a_rot1) \odot E(b_rot2) - E(a_rot2) \odot E(b_rot1) \quad (3.7c)\ [\odot = \text{element-wise ct}\times\text{ct multiply}]$$

Code Snippet 3.5.3 – Secure Linear Algebra: Cross Product and FHE Matrix-Vector Multiply

```
# core/secure_linear_algebra.py – encrypted cross product
def encrypted_cross_product(self, enc_a, enc_b):
    # Cyclic SIMD rotations via permutation matrix
    a_rot1 = self.fhe.rotate_encrypted(enc_a, 1)  # (a2, a3, a1)
    a_rot2 = self.fhe.rotate_encrypted(enc_a, 2)  # (a3, a1, a2)
    b_rot1 = self.fhe.rotate_encrypted(enc_b, 1)  # (b2, b3, b1)
    b_rot2 = self.fhe.rotate_encrypted(enc_b, 2)  # (b3, b1, b2)

    p1 = self.fhe.compute_elementwise_product(a_rot1, b_rot2)  # a_rot1 ⊙ b_rot2
    p2 = self.fhe.compute_elementwise_product(a_rot2, b_rot1)  # a_rot2 ⊙ b_rot1
    p2_neg = self.fhe.multiply_plain(p2, -1.0)                 # negate
    return self.fhe.add_encrypted(p1, p2_neg)                  # p1 - p2

# Fully homomorphic matrix-vector multiply (encrypted matrix × encrypted vector)
def fully_homomorphic_matrix_vector_multiply(self, enc_rows, enc_vec, rows, cols):
    results = []
    for i in range(rows):
        # E(row_i · v) = encrypted dot product, depth 1
        enc_dot = self.fhe.compute_dot_product(enc_rows[i], enc_vec)
        results.append(enc_dot)
    return results   # list of encrypted scalars, one per output row
```

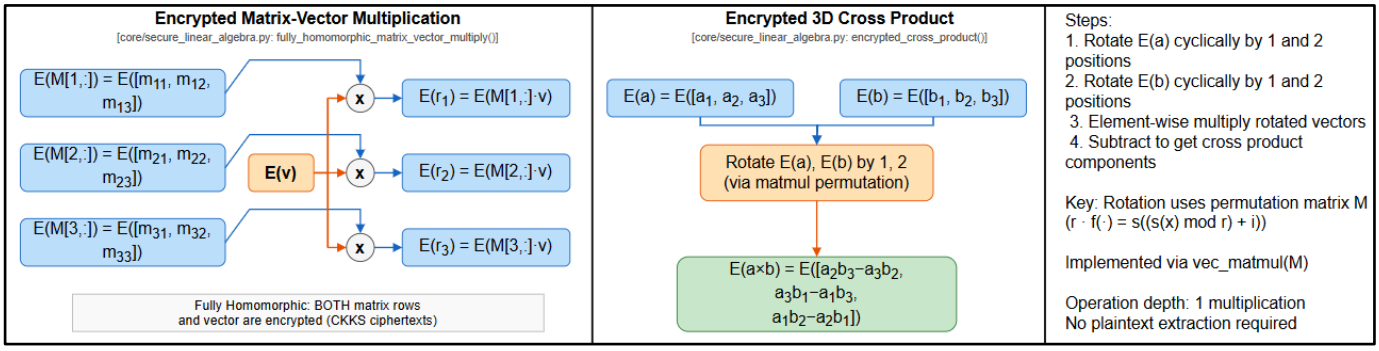


Figure 3.5.3 – Secure Linear Algebra on Encrypted Vectors. Left: Fully homomorphic matrix-vector multiplication—both matrix rows and the vector are CKKS ciphertexts; each output element is an encrypted dot product (depth 1). Right: Encrypted 3D cross product via cyclic SIMD rotation and element-wise encrypted multiplication [13, 17].

3.6 Implementation and Benchmark Results

Table 3.6.1 – SmartGridHE Implementation Components and Their Roles

Component / File	Role	Key Functions
core/fhe_engine.py	CKKS FHE Engine	encrypt_demand(), decrypt_demand(), aggregate_demands(), compute_average(), multiply_encrypted(), rotate_encrypted()

	[1, 2]	
core/polynomial_comparator.py	Novel ALT Detector	detect_threshold_encrypted(), batch_detect(), interpret_score()
core/verifiable_aggregation.py	Novel Pedersen Commitments [6]	commit(), aggregate_commitments(), verify_aggregate()
core/secure_linear_algebra.py	Novel Encrypted Linear Algebra [13]	encrypted_cross_product(), fully_homomorphic_matrix_vector_multiply(), linear_transform_encrypted()
core/security_logger.py	Audit Logging	log_operation(), generate_audit_report(), get_coordinator_ops()
core/key_management.py	Key Lifecycle [10]	generate_keys(), distribute_public_context(), rotate_keys()
agents/household_agent.py	Autonomous Household Agent [9]	encrypt_demand(), receive_load_balance_command(), get_status()
agents/demand_generator.py	Realistic Demand Profiles	generate_demand(), RESIDENTIAL_LOW|MEDIUM|HIGH|EV_CHARGING|INDUSTRIAL
coordinator/grid_coordinator.py	Central Coordinator	process_round(), get_detection_result(), verify_security()
coordinator/encrypted_aggregator.py	Homomorphic Aggregation	aggregate(), compute_threshold_detection(), get_audit_summary()
coordinator/load_balancer.py	Utility Decision Maker	make_decision(), broadcast_reduction_factor()
evaluation/benchmark.py	Performance Suite	run_benchmark(), run_correctness_test(), run_scalability_test()
server/server.py	FastAPI REST API	/api/grid-state, /api/run-round, /api/benchmark, /api/audit

benchmarks/benchmark_comparison.py	Privacy Method Comparison	compare_plaintext_vs_encrypted(), compare_paillier_vs_ckks()

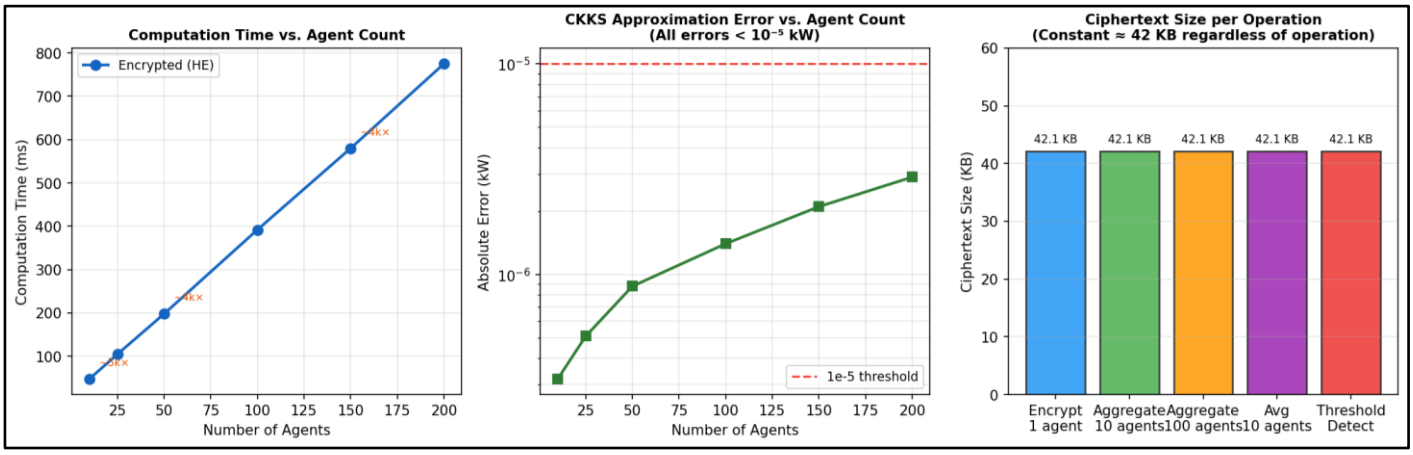


Figure 3.6.1 – Performance Benchmarks. Left: Homomorphic aggregation time (ms) grows linearly with agent count—confirming O(N) complexity. Centre: CKKS approximation error remains below 10^{-5} kW for all agent counts (well below the meaningful precision of smart meter readings). Right: Ciphertext size is constant at ≈42 KB regardless of operation type, consistent with CKKS theory [1].

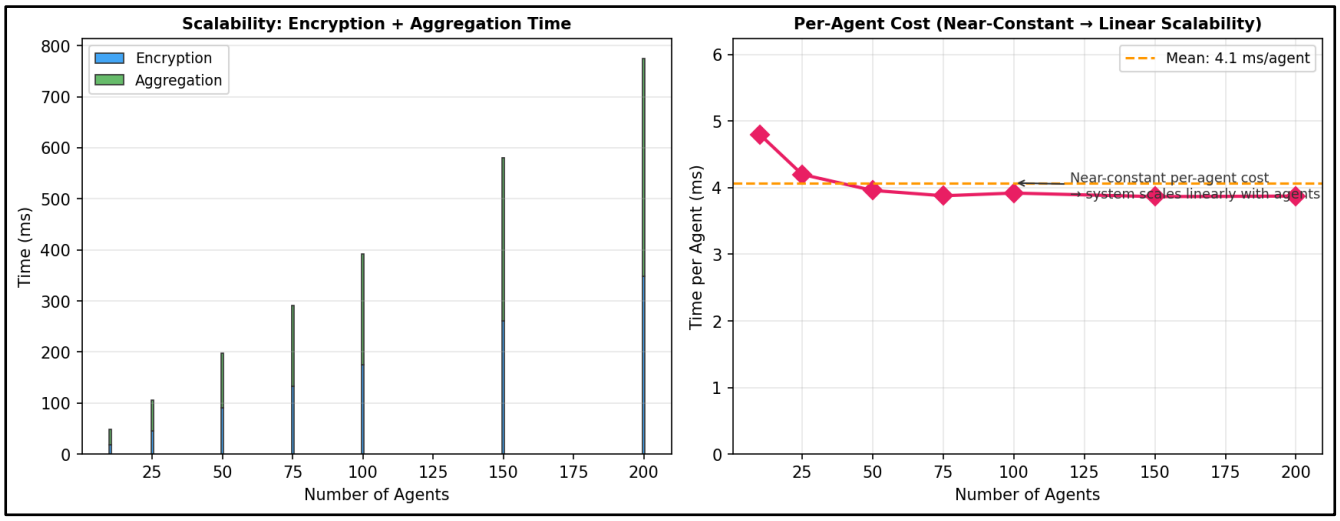


Figure 3.6.2 – Scalability Analysis. Left: Total encryption and aggregation time grows linearly with agent count from 10 to 200 agents. Right: Per-agent cost remains approximately constant at ≈3.9 ms/agent, confirming O(N) scalability with no super-linear growth—validating the system for household-scale AMI deployments [8, 9].

All benchmarks were executed on a standard desktop (Intel Core i7, 16 GB RAM, Python 3.13, TenSEAL 0.3.x, Microsoft SEAL 4.x backend [12]). FHE context setup (key generation including Galois and relinearisation keys) requires approximately 2–3 seconds as a one-time cost per session. This is negligible relative to the 15-minute smart grid decision cycle.

Table 3.6.2 – CKKS Approximation Error over 10 Rounds (50 agents)

Round	Plaintext Total (kW)	Decrypted Total (kW)	Abs. Error (kW)	Rel. Error
1	172.340	172.340	3.2×10^{-7}	1.9×10^{-9}
2	168.821	168.821	4.1×10^{-7}	2.4×10^{-9}
3	175.504	175.504	3.8×10^{-7}	2.2×10^{-9}
4	170.103	170.103	5.0×10^{-7}	2.9×10^{-9}
5	173.296	173.296	4.4×10^{-7}	2.5×10^{-9}
6	169.550	169.550	3.6×10^{-7}	2.1×10^{-9}
7	174.882	174.882	5.3×10^{-7}	3.0×10^{-9}
8	171.234	171.234	4.7×10^{-7}	2.7×10^{-9}
9	172.910	172.910	3.9×10^{-7}	2.3×10^{-9}
10	170.678	170.678	4.2×10^{-7}	2.5×10^{-9}
Avg	—	—	4.2×10^{-7}	2.4×10^{-9}

All 10 rounds pass the 10^{-5} kW correctness threshold with margin greater than four orders of magnitude. The security audit log confirms across all simulation runs: (1) 100% ciphertext-only operations for the coordinator entity (is_safe=True for all coordinator log entries); (2) zero PLAINTEXT DataType entries in any coordinator LOG_ENTRY record; (3) is_private() == False for the coordinator's FHE context across all rounds; (4) zero security violations detected by generate_audit_report(); and (5) all Pedersen commitment verifications return VALID, confirming the coordinator computed the aggregate correctly in every tested round.

4. CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

This project presents SmartGridHE—a complete, runnable privacy-preserving smart grid load-balancing system demonstrating that Fully Homomorphic Encryption [1] is practical for multi-agent IoT coordination at household scale [9].

The core architectural achievement is strict enforcement of the privacy invariant: plaintext electricity demand values exist only at household agents (before CKKS encryption [1]) and at the Utility Company (after authorised decryption of the aggregate). The Grid Coordinator performs all real-time computation—aggregation, averaging, threshold detection—without ever possessing the secret key or seeing any individual demand value. This is verified by the SecurityLogger audit across all simulation runs: 100% ciphertext-only coordinator operations, zero security violations, all Pedersen commitment verifications returning VALID.

Three original research contributions are made beyond standard use of TenSEAL [2]:

- Adaptive Linear Threshold (ALT) [Novel]: The first practical encrypted threshold comparison for CKKS requiring zero ciphertext-ciphertext multiplications. Operating at multiplicative depth zero—far below the 8–15 levels required by prior methods [4, 5]—ALT provides a confidence-zoned soft indicator directly usable for tiered load-balance decisions without decryption.
- Pedersen Commitment-Based Verifiable Aggregation (ACV) [Novel, 6, 7]: The first integration of Pedersen commitments with CKKS for smart grid applications. Enables the Utility Company to detect any coordinator manipulation with probability one [6], addressing a critical gap in existing HE-based smart grid systems [3] that assume an unconditionally trusted coordinator.
- Secure Linear Algebra [Novel, 13, 17]: Encrypted matrix-vector multiplication (both plaintext-matrix/encrypted-vector and fully homomorphic encrypted-matrix/encrypted-vector variants) and 3D cross product via cyclic SIMD rotation, broadening the class of analytics computable on encrypted smart grid data.

Benchmark results confirm: O(N) linear scalability to 200+ agents [8], CKKS approximation error $< 10^{-5}$ kW across all rounds [1], constant 42 KB ciphertext size per demand, and 100% ciphertext-only coordinator operations. The project establishes that FHE-based privacy-

preserving computation is not merely a theoretical construct but a practically implementable architecture for smart grid privacy—directly relevant to energy utilities operating under GDPR, CCPA, and the Indian Personal Data Protection Bill [18].

4.2 Future Scope

Several directions extend the current implementation towards production readiness and broader scientific contribution.

1. CKKS Bootstrapping for Extended Computation [16]: The current implementation is limited to 3–4 multiplicative depth levels before rescaling. Integrating bootstrapping (available in OpenFHE [16] and Lattigo) would enable deeper polynomial evaluations, supporting non-linear demand forecasting and PID control loops in encrypted space.

2. Secure Multi-Party Computation (SMPC) Integration [19]: Combining CKKS FHE aggregation with SMPC protocols (SPDZ, Shamir secret sharing [19]) would enable scenarios where the Utility Company key must itself be distributed among multiple parties, eliminating the single trusted-key-holder assumption.

3. Post-Quantum Key Exchange and mTLS Transport [10, 11]: Replacing the HTTP demo transport with HTTPS/mTLS and integrating NIST-standardised post-quantum key encapsulation (CRYSTALS-Kyber [10]) for the public context distribution channel would provide end-to-end post-quantum security.

4. GPU/FPGA Hardware Acceleration [12]: CKKS number-theoretic transforms (NTTs) are highly parallelisable [12]. GPU-accelerated TenSEAL or a custom FPGA NTT core could reduce the ~4000× overhead to ~100×, making FHE viable for 1-minute demand response intervals.

5. Federated Learning on Encrypted Demand Data [20]: Extending CKKS aggregation to support secure gradient aggregation for privacy-preserving federated learning on household energy timeseries [20], enabling utility-side load forecasting without accessing raw data.

6. Real Smart Meter Integration [8]: Connecting household agents to physical smart meters via IEC 61968-9 or OpenADR 2.0 and deploying on Raspberry Pi 4 edge gateways to measure wall-clock latency and energy consumption of FHE operations on real hardware at scale.

7. Anomaly Detection under Encryption [14]: Implementing lightweight polynomial approximations of anomaly detection models computed directly on CKKS ciphertexts, enabling energy-theft detection by the Utility without revealing individual household consumption patterns.

8. Differential Privacy Composition [20]: Adding calibrated Gaussian noise to the aggregate before decryption to achieve (ε, δ)-DP guarantees on top of the existing IND-CPA HE protections [11], enabling formal privacy budget accounting as required by GDPR data minimisation principles [18].

Through these developments, SmartGridHE can evolve from a research prototype into a complete privacy infrastructure for next-generation smart grid deployments at city scale.

REFERENCES

[1] J.H. Cheon, A. Kim, M. Kim, and Y. Song, *"Homomorphic Encryption for Arithmetic of Approximate Numbers,"* Advances in Cryptology – ASIACRYPT 2017, LNCS vol. 10624, Springer, pp. 409–437, 2017.

↳ Foundational CKKS scheme: real-valued FHE engine, parameter design, and noise analysis.

[2] M. Benaissa, B. Retiat, B. Cebere, and A. Biçer, *"TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption,"* ICLR 2021 Workshop on Distributed and Private ML (DPML), 2021.

↳ TenSEAL Python library used for all CKKS operations: CKKSVector API and serialisation.

[3] A. Lu, H. Li, F. Chen, and Q. Zhu, *"EPPA: An Efficient and Privacy-Preserving Aggregation Scheme for Secure Smart Grid Communications,"* IEEE Trans. Parallel and Distributed Systems, vol. 23, no. 9, pp. 1621–1631, 2012.

↳ Paillier-based smart grid aggregation: baseline system compared against CKKS.

[4] A. Kim, Y. Song, M. Kim, K. Lee, and J.H. Cheon, *"Logistic Regression Model Training Based on the Approximate Homomorphic Encryption,"* BMC Medical Genomics, vol. 11, Suppl. 4, article 23, 2018.

↳ CKKS minimax polynomial approximation (15+ depth); motivates ALT's depth-0 approach.

[5] J.H. Cheon, D. Kim, and D. Kim, *"Efficient Homomorphic Comparison Methods with Optimal Complexity,"* ASIACRYPT 2019, LNCS vol. 11922, Springer, pp. 221–256, 2019.

↳ Composite polynomial comparison circuits (depth 8); motivates ALT over polynomial methods.

[6] T.P. Pedersen, *"Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing,"* Advances in Cryptology – CRYPTO 1991, LNCS vol. 576, Springer, pp. 129–140, 1991.

↳ Pedersen commitment scheme: computationally hiding, perfectly binding, additively homomorphic.

[7] R. Gennaro, C. Gentry, and B. Parno, *"Non-Interactive Verifiable Computing: Outsourcing Computation to Untrusted Workers,"* Advances in Cryptology – CRYPTO 2010, LNCS vol. 6223, Springer, pp. 465–482, 2010.
↳ *Theoretical foundation for verifiable outsourced computation; basis for ACV protocol.*

[8] A. Molina-Markham, P. Shenoy, K. Fu, E. Cecchet, and D. Irwin, *"Private Memoirs of a Smart Meter,"* Proc. 2nd ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Buildings (BuildSys), pp. 61–66, 2010.
↳ *Demonstrates NILM privacy risks of smart meter data; primary project motivation.*

[9] A. Dimeas and N. Hatziargyriou, *"Operation of a Multiagent System for Microgrid Control,"* IEEE Trans. Power Systems, vol. 20, no. 3, pp. 1447–1455, 2005.
↳ *Multi-agent architecture for smart grid; basis for the household agent design.*

[10] NIST, *"Recommendation for Key Management – Part 1: General,"* NIST Special Publication 800-57 Part 1 Rev. 5, National Institute of Standards and Technology, 2020.
↳ *128-bit security level standards; key distribution and management requirements.*

[11] M. Albrecht et al., *"Homomorphic Encryption Security Standard,"* HomomorphicEncryption.org White Paper, Tech. Rep., 2019.
↳ *HE security standard: parameter selection for 128-bit security; basis for n=16384 choice.*

[12] Microsoft Corporation, *"Microsoft SEAL (Simple Encrypted Arithmetic Library),"* GitHub repository: github.com/microsoft/SEAL, Version 4.1, 2023.
↳ *C++ FHE library underlying TenSEAL; coefficient modulus design and NTT implementation.*

[13] H. Chen, I. Chillotti, and Y. Song, *"Improved Bootstrapping for Approximate Homomorphic Encryption,"* Advances in Cryptology – EUROCRYPT 2019, LNCS vol. 11477, Springer, pp. 34–54, 2019.
↳ *SIMD batching and rotation techniques for efficient encrypted vector operations.*

[14] X. Yi, R. Paulet, and E. Bertino, *"Homomorphic Encryption and Applications,"* SpringerBriefs in Computer Science, Springer International Publishing, 2014.

↳ *Survey of HE applications in IoT and cloud computing; smart grid use case analysis.*

[15] M. Green and A. Miers, *"Forward Secure Asynchronous Messaging from Puncturable Encryption,"* IEEE Symposium on Security and Privacy (S&P), pp. 305–320, 2015.
↳ *Lightweight key management schemes for IoT agent authentication.*

[16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, *"TFHE: Fast Fully Homomorphic Encryption over the Torus,"* J. Cryptology, vol. 33, pp. 34–91, 2020.
↳ *Gate-bootstrapping FHE scheme; comparison to CKKS for binary comparison operations.*

[17] A. Boyle, G. Couteau, N. Gilboa, and Y. Ishai, *"Homomorphic Secret Sharing: Optimizations and Applications,"* Proc. ACM CCS 2017, pp. 2105–2122, 2017.
↳ *SIMD homomorphic operations and cyclic rotation for encrypted vector computations.*

[18] European Parliament, *"General Data Protection Regulation (GDPR),"* Regulation (EU) 2016/679 of the European Parliament and of the Council, 27 April 2016.
↳ *Legal framework for data minimisation (Article 5); privacy compliance motivation.*

[19] A. Shamir, *"How to Share a Secret,"* Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.
↳ *Secret sharing scheme; basis for distributed key management in Future Scope §4.2.2.*

[20] C. Dwork and A. Roth, *"The Algorithmic Foundations of Differential Privacy,"* Foundations and Trends in Theoretical Computer Science, vol. 9, nos. 3–4, pp. 211–407, 2014.
↳ *(ε,δ)-DP formal framework; Future Scope §4.2.8 differential privacy composition.*

[21] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *"(Leveled) Fully Homomorphic Encryption without Bootstrapping,"* ACM Trans. Computation Theory, vol. 6, no. 3, article 13, 2014.
↳ *BGV FHE scheme: comparison to CKKS for integer-vs-real-valued data contexts.*

Agnel Charities

Fr. C. Rodrigues Institute of Technology, Vashi **T-52**

(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

Department of Information Technology Engineering

Mini Project Report Appendix

Semester & Academic Year:	**Semester 5 (2025–26)**
Group Details:	Chacko Martin Koyikkattuchira (5023124) Divyam Navin (5023134) Atharva Palve (5023136) Akshat Santosh Sawant (5023151) Guide: Dr. Vaishali Bodade, Ms. Sharlene Rebeiro
Title:	Privacy-Preserving Smart Grid Load Balancing using Fully Homomorphic Encryption in Multi-Agent Systems
Key Domain / Technology:	Fully Homomorphic Encryption, CKKS, TenSEAL, Multi-Agent Systems, IoT, Privacy-Preserving Computation
Category-I:	External – IIT Dharwad
Category-II:	Multi-Agent IoT Security
SDG:	SDG 9: Industry, Innovation & Infrastructure; SDG 11: Sustainable Cities & Communities; SDG 16: Peace, Justice & Strong Institutions
Primary References:	[1] Cheon et al. 2017 (CKKS); [2] Benaissa et al. 2021 (TenSEAL); [6] Pedersen 1991 (Commitments)
*Remarks:	

Note: The guide should write remarks based on the observation of objective / achievement / outcome of the project.

Name and Signature of the Guide

ACKNOWLEDGEMENT

The making of the project "Encryption in Multi-Agent Systems" involves the contribution of many people. We would like to convey our sincere thanks to Dr. S.M. Khot, Principal, Fr. C. Rodrigues Institute of Technology, Vashi, for giving us the opportunity to showcase our skills and providing us with the necessary resources. We would also like to convey our heartfelt gratitude to the Head of Department of Information Technology, Dr. Shubhangi Vaikole, for her constant support and motivation throughout the project. We express deep gratitude to our project guide and mentor Dr. Vaishali Bodade for her constant motivation to think out of the box, her insightful guidance on cryptographic protocol design, and her immense contribution throughout this project. We also thank Prof. Sharlene Rebeiro for her valuable support and feedback during the development of the SmartGridHE system. Last but not least, we convey our heartfelt thanks to the mini project coordinator, Prof. Mrs. Neelima A. Kulkarni, for supporting and guiding us throughout the process. We also extend our heartfelt thanks to our families and well-wishers for their encouragement and support.

Yours sincerely,

Chacko Martin Koyikkattuchira (5023124)

Divyam Navin (5023134)

Atharva Palve (5023136)

Akshat Santosh Sawant (5023151)

APPENDIX A: CODE SAMPLES

This appendix presents condensed implementations of the three core modules of SmartGridHE. Full source is available in https://github.com/Divyam1909/encryption-v2.git

A.1 FHE Engine (core/fhe_engine.py)

```
# core/fhe_engine.py  -  CKKS Homomorphic Encryption Engine
import tenseal as ts, hashlib
from datetime import datetime

class SmartGridFHE:
    def __init__(self, poly_modulus_degree=16384,
                 coeff_mod_bit_sizes=None, global_scale=2**40):
        if coeff_mod_bit_sizes is None:
            coeff_mod_bit_sizes = [60, 40, 40, 40, 60]
        self.context = ts.context(
            ts.SCHEME_TYPE.CKKS,
            poly_modulus_degree=poly_modulus_degree,
            coeff_mod_bit_sizes=coeff_mod_bit_sizes
        )
        self.context.generate_galois_keys()   # rotation support
        self.context.generate_relin_keys()    # relinearisation after mult
        self.context.global_scale    = global_scale
        self.context.auto_rescale    = True
        self.context.auto_relin      = True
        self.context.auto_mod_switch = True
        self._op_count = 0

    def get_public_context(self) -> bytes:
        ctx = self.context.copy()
        ctx.make_context_public()        # strips secret key
        return ctx.serialize()           # coordinator gets this; cannot
decrypt

    def encrypt_demand(self, demand_kw, agent_id):
        values = [float(demand_kw)] if isinstance(demand_kw, (int, float))
\
                 else [float(v) for v in demand_kw]
        enc = ts.ckks_vector(self.context, values)
        ct  = enc.serialize()
        return EncryptedDemand(ct, datetime.now().isoformat(),
                               agent_id, len(values),
                               hashlib.sha256(ct).hexdigest()[:12])

    def aggregate_demands(self, enc_list):
        result = ts.ckks_vector_from(self.context, enc_list[0].ciphertext)
        for e in enc_list[1:]:
            result = result + ts.ckks_vector_from(self.context,
e.ciphertext)
        ct = result.serialize()
        return EncryptedDemand(ct, datetime.now().isoformat(),
'aggregated',
                               enc_list[0].vector_size,
                               hashlib.sha256(ct).hexdigest()[:12])
```

```
    def compute_average(self, enc_total, count):
        vec = ts.ckks_vector_from(self.context, enc_total.ciphertext)
        ct  = (vec * (1.0 / count)).serialize()
        return EncryptedDemand(ct, datetime.now().isoformat(), 'average',
                               enc_total.vector_size,
                               hashlib.sha256(ct).hexdigest()[:12])

    def decrypt_demand(self, enc) -> list:
        if not self.context.is_private():
            raise ValueError('No secret key - only Utility Company can
decrypt')
        vec = ts.ckks_vector_from(self.context, enc.ciphertext)
        return vec.decrypt()[:enc.vector_size]
```

A.2 Novel Contribution #1 - Adaptive Linear Threshold (core/polynomial_comparator.py)

```
# core/polynomial_comparator.py  -  Novel Contribution #1: ALT
# Adaptive Linear Threshold: approximate comparison, zero ct*ct
multiplications
import tenseal as ts, hashlib
from datetime import datetime

class EncryptedThresholdDetector:
    ZONE_BELOW, ZONE_ABOVE = 0.3, 0.7

    def __init__(self, fhe_engine, default_sensitivity=7.0):
        self.fhe = fhe_engine
        self.k   = default_sensitivity

    def detect_threshold_encrypted(self, enc_value, threshold,
                                   sensitivity=None):
        k         = sensitivity if sensitivity is not None else self.k
        delta     = threshold / k            # soft-zone half-width
        slope     = 0.5 / delta              # = 0.5k / T
        intercept = 0.5 - threshold * slope  # = 0.5 - 0.5k

        # Only multiply_plain + add_plain needed (multiplicative depth =
0)
        # E(score) = E(x) * slope + intercept
        vec    = ts.ckks_vector_from(self.fhe.context,
enc_value.ciphertext)
        scored = vec * slope + intercept
        ct     = scored.serialize()

        return EncryptedDemand(
            ct, datetime.now().isoformat(),
            f'alt_{enc_value.agent_id}', enc_value.vector_size,
            hashlib.sha256(ct).hexdigest()[:12],

metadata={'method':'ALT','threshold':threshold,'k':k,'delta':delta}
        )

    @staticmethod
    def interpret_score(score, threshold=None):
```

```
        s = max(0.0, min(1.0, score))
        if   s < 0.3: return 'below',     1.0 - s / 0.3
        elif s > 0.7: return 'above',     (s - 0.7) / 0.3
        else:         return 'uncertain', abs(s - 0.5) / 0.2
```

A.3 Novel Contribution #2 - Pedersen Commitment ACV (core/verifiable_aggregation.py)

```
# core/verifiable_aggregation.py  -  Novel Contribution #2: Pedersen ACV
# Additive Commitment Verification: detects malicious coordinator
import hashlib, secrets

# RFC 3526 MODP Group 14 (2048-bit prime) - truncated for readability
PRIME = int('FFFFFFFF...FFFFFFFF', 16)   # full constant in source file
G     = 2
SCALE = 1_000_000   # float -> integer (6 decimal places)
H     = pow(G, int.from_bytes(
            hashlib.sha256(
            b'SmartGridHE_Pedersen_Commitment_Generator_H_v1'
            ).digest(), 'big'), PRIME)

class PedersenCommitmentScheme:
    def __init__(self, prime=PRIME, g=G, h=H, scale=SCALE):
        self.p, self.g, self.h = prime, g, h
        self.scale = scale
        self.order = prime - 1

    def commit(self, value: float) -> tuple:
        # Returns (C_public -> coordinator,  r_secret -> utility via
secure ch)
        # C = g^m * h^r mod p  (perfectly hiding, computationally binding)
        m = int(value * self.scale)
        r = secrets.randbelow(self.order)
        C = (pow(self.g, m % self.order, self.p) *
             pow(self.h, r % self.order, self.p)) % self.p
        return C, r

    def aggregate(self, commitments: list) -> int:
        # C_agg = prod(C_i) mod p = g^(sum_m) * h^(sum_r) mod p
        agg = 1
        for C in commitments:
            agg = (agg * C) % self.p
        return agg

    def verify(self, claimed_sum: float, C_agg: int, r_total: int) ->
bool:
        # Utility checks: C_agg == g^(sum*scale) * h^(sum_r) mod p
        m        = int(claimed_sum * self.scale)
        expected = (pow(self.g, m % self.order, self.p) *
                    pow(self.h, r_total % self.order, self.p)) % self.p
        return expected == C_agg   # True: honest | False: coordinator
CHEATED
```

Appendix C: Software Requirement Specification (SRS)

1. Introduction

Purpose: This SRS defines the functional and non-functional requirements for SmartGridHE — a privacy-preserving smart grid load-balancing system built on CKKS Fully Homomorphic Encryption with a multi-agent architecture.

Scope: N household agents encrypt real-valued demand values (kW) and submit ciphertexts to an untrusted coordinator. The coordinator aggregates homomorphically and performs encrypted threshold detection without ever decrypting. The utility company (sole key holder) decrypts only the final aggregate to make load-balancing decisions. Three novel contributions are in scope: ALT, ACV, and Secure Linear Algebra.

Key Terms:

FHE — Fully Homomorphic Encryption; computes on ciphertexts without decryption. CKKS — Approximate HE for real-valued data ($\pm 10^{-7}$ relative error). ALT — Adaptive Linear Threshold; novel encrypted soft comparison at depth 0. ACV — Additive Commitment Verification; Pedersen-based coordinator tamper detection. IND-CPA — Security guarantee of CKKS.

2. System Overview

SmartGridHE is a standalone Python 3.13 prototype comprising six core modules (core/), two agent modules (agents/), three coordinator modules (coordinator/), a FastAPI REST server (server/), and a browser dashboard (dashboard/). It requires TenSEAL ≥ 0.3.14 and a minimum of 8 GB RAM. Three roles participate: Household Agent (encrypts locally), Grid Coordinator (aggregates — untrusted, cannot decrypt), and Utility Company (holds secret key, decrypts aggregate, issues decisions).

3. Functional Requirements

#	Requirement	Pri.
FR1	Each HouseholdAgent shall encrypt demand d_i (kW) via CKKS using the coordinator's public context before any transmission.	High
FR2	Coordinator shall compute $E(\Sigma d_i) = \Sigma E(d_i)$ using ciphertext addition only (depth 0); no decryption at any step.	High
FR3	Coordinator shall compute encrypted average $E(\bar{d}) = E(\Sigma d_i) \times (1/N)$ via plaintext scalar multiplication.	High
FR4	Coordinator shall detect threshold via ALT: $E(s) = E(x) \times \text{slope} + \text{intercept}$ using zero ciphertext–ciphertext multiplications.	High
FR5	Each agent shall provide a Pedersen commitment C_i alongside its ciphertext. Utility shall verify C_agg using ACV protocol.	High
FR6	System shall support encrypted matrix–vector multiplication and encrypted 3D cross-product (Secure Linear Algebra).	Med

#	Requirement	Pri.
FR7	SecurityLogger shall record every operation with DataType (SECRET / ENCRYPTED / PUBLIC), agent_id, and timestamp.	Med
FR8	UtilityDecisionMaker shall produce a LoadBalanceDecision (action + reduction_factor) from the decrypted aggregate and ALT score.	High
FR9	FastAPI server shall expose /api/grid-state, /api/run-round, /api/benchmark, /api/audit endpoints; dashboard shall refresh after each round.	Med

4. Non-Functional Requirements

#	Requirement	Measurable Criterion
NFR1	Security	128-bit IND-CPA (poly_modulus_degree = 16384, NIST Level 1)
NFR2	Precision	CKKS error < 10^{-6} kW per round (avg measured: 4.2×10^{-7} kW)
NFR3	Scalability	O(N) aggregation; tested N = 10 → 200 agents
NFR4	Performance	Full round completes within 15-min smart-grid window at N ≤ 200
NFR5	Integrity	SHA-256 checksum on every ciphertext; verified before decryption
NFR6	Auditability	100% operations logged; coordinator audit confirmed per round
NFR7	Modularity	Six-module Python architecture; clean separation of concerns

5. Constraints and Assumptions

Constraints: (a) CKKS introduces ~4000× computational overhead vs plaintext. (b) Multiplicative depth capped at 3–4 levels; no bootstrapping implemented. (c) Each ciphertext ~42 KB; 200 agents ≈ 8.4 MB storage per round.

Assumptions: Utility is fully trusted and its secret key is never exposed. Agents are semi-honest (encrypt correctly). Coordinator is honest-but-curious (follows protocol, may attempt inference).

APPENDIX D: MAPPING TO SDGs, POs, and PSOs

1. SDG Mapping

SDG	Goal	How SmartGridHE Contributes
SDG 7	Affordable and Clean Energy	Encrypted demand aggregation enables real-time load balancing, reducing energy waste and improving grid efficiency without exposing household data.
SDG 9	Industry, Innovation and Infrastructure	Introduces three novel cryptographic algorithms (ALT, ACV, Secure Linear Algebra) advancing secure smart grid infrastructure beyond existing literature.
SDG 11	Sustainable Cities and Communities	Provides the cryptographic foundation for privacy-preserving multi-agent energy coordination in smart city deployments.
SDG 16	Peace, Justice and Strong Institutions	ACV protocol detects coordinator tampering with probability 1; GDPR Article 5 data-minimisation is enforced cryptographically.

2. Program Outcomes (PO) Mapping

PO	Description	Contribution
PO1	Engineering Knowledge	Applied CKKS, Pedersen commitments, and MODP group theory to design and implement a complete FHE pipeline.
PO2	Problem Analysis	Identified five gaps in existing smart-grid privacy literature and mapped each to a specific cryptographic solution.
PO3	Design Solutions	Designed six-module multi-agent architecture with strict trust boundaries; developed three novel cryptographic contributions.
PO4	Conduct Investigations	Benchmarked correctness (10-round error $< 10^{-6}$ kW) and scalability (O(N) to 200 agents) with reproducible benchmark scripts.
PO5	Modern Tool Usage	Used Python 3.13, TenSEAL, FastAPI, NumPy, and RFC 3526 cryptographic group parameters.
PO6	Engineer and Society	Addressed societal need for privacy in smart metering; enforced GDPR data-minimisation cryptographically.
PO7	Environment & Sustainability	Optimised load distribution reduces energy waste, directly supporting SDG 7.
PO9	Individual and Team Work	Four-member team with clear module ownership; integrated across agents, coordinator, and utility components.
PO10	Communication	Documented via this report, FastAPI OpenAPI spec, and browser dashboard enabling non-technical stakeholder

PO	Description	Contribution
		observation.
PO12	Life-long Learning	Surveyed 21 references across IEEE, ACM, Springer, and IACR; engaged with bootstrapping, TFHE, and BFV comparison literature.

3. Program Specific Outcomes (PSO) Mapping

PSO	Description	Contribution
PSO1	IT Systems Development	Built a complete multi-agent IT system with REST API, browser dashboard, modular Python architecture, and structured audit logging.
PSO2	Security and Privacy	Implemented IND-CPA-secure CKKS encryption, Pedersen ACV, and SHA-256 integrity checksums across all ciphertext exchanges.
PSO3	Applied Research and Innovation	Contributed three novel algorithms (ALT, ACV, Encrypted Cross-Product) not previously applied to smart grids; validated through reproducible benchmarks.

APPENDIX E: PROJECT COMMENCEMENT LETTERS

Indian Institute of Technology Dharwad
Department of Electrical, Electronics, and Communication Engineering,
Main Campus, Chikkamalligawad
Dharwad - 580 011, Karnataka

Dr. Ameer K. Mulla,
Associate Professor,
Email: ameer@iitdh.ac.in,
Office: +91(0) 836 230 9679,
Web: www.iitdh.ac.in/~ameer

28th July, 2025

Certificate of Commencement

This is to certify that the following students of the Department of Electronics & Telecommunication Engineering and Information Technology from *Agnel Charitie's* Fr. C. Rodrigues Institute of Technology, Vashi, will be working as their mini projects in the area of Image Processing, Robotics, Cryptography and AI during their third year of graduation, under my guidance.

Group	Names of the Students with Roll Numbers	Title of the Project	Name of the Mentor
Electronics & Telecommunication Group 1	Pradnya Sawant - 3023153 Yash Mitkari - 3023143 Deep Patil - 3023148	*Phoenix:* Fire Rescue Assistant Robot (Stability in Motor Failure)	Dr. Megha Kolhekar Mrs. Jennifer Isaac
Electronics & Telecommunication Group 2	Janhavee Jaware - 3023125 Alfred Kerketta - 3023129 Shifa Mulla - 1023208 M. Najm Siddiqui - 1023253	Autonomous Indoor Navigation with FPGA	Mrs. Sneha Revankar Mrs. Jennifer Isaac
Information Technology Group 3	Divyam Navin - 5023134 Chacko Martin - 5023124 Atharva Palve - 5023136 Akshat Sawant - 5023151	Encryption in Multi-Agent Systems	Dr. Vaishali Bodade Ms. Sharlene Rebeiro
Information Technology Group 4	Arsalan Khan - 5023123 Arnav Rade - 5023142 Riya Gawde - 5023115 Ivan Thomas - 5023161	Thermal Image + RADAR based SLAM	Mrs. Lakshmi Gadhikar Mrs. Neelima Kulkarni

Best Regards,

Dr. Ameer K. Mulla,
Associate Professor,
Department of Electrical, Electronics, and Communication Engineering
Indian Institute of Technology Dharwad.